



Samsung's Tiger strategy

Samsung's TIGER strategy involves narrowing the gap between themselves and Apple.
<https://digit.in/mar22-41>



Samsung attempts to go green

Samsung's upcoming Galaxy devices will be made partly from recycled fishing nets.
<https://digit.in/mar22-42>

```

✓ ~/code
11:53 $ mkdir eth-todo-list
✓ ~/code
11:53 $ cd eth-todo-list/
✓ ~/code/eth-todo-list
11:53 $ truffle version
Truffle v5.0.2 (core: 5.0.2)
Solidity v0.5.0 (solc-js)
Node v8.9.0
✓ ~/code/eth-todo-list
11:54 $ truffle init

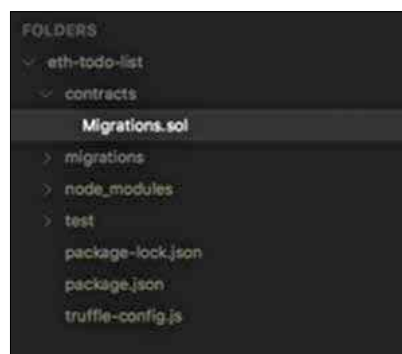
✓ Preparing to download
✓ Downloading
✓ Cleaning up temporary files
✓ Setting up box

Unbox successful. Sweet!
```

`json` file initialised for use. Hence, you have to manually create the file and include the following dependencies, following the same structure: <https://pastebin.com/T6DKnFHB>

Let's break down the `devDependencies{...}` section. `bootstrap` is a library used for the frontend of the application. `chai` is a testing library used to perform tests on smart contracts. `lite-server` is used to launch a local server in your system. `nodemon` is a package used to track changes in the project and apply it to the locally deployed version without having to restart the server every time. Lastly `truffle`, as we have seen before, is the general framework used for solidity, to write and compile smart contracts. Now that you have initialised these dependencies, it is time to install them! Use `npm install` to automatically install the dependencies from `package.json`

After installing everything, your folder structure should look like this:



The `Migrations.sol` file is a pre-built set of smart contracts that connects your project to a blockchain network. It already contains some very useful code. The "contracts" folder is where you will write all your smart contracts. To start writing your smart contracts for the to-do list, create a file called `TodoList.sol` inside the contracts folder. Inside this file, we will write all the code necessary for the logic of the to-do list.

WRITING YOUR FIRST SMART CONTRACT

In the `TodoList.sol` file is where you will write your first smart contract. The first step is to mention the language that you are using at the start of the file. It can be done via this command: `pragma solidity ^0.5.0;`

Solidity almost uses a javascript-like language structure, and all the other functions and contracts adhere to that format. Of course, there are differences in the types of variables, their scopes, which we can explore as we go deeper into the project.

In solidity, one deployable unit of code is known as contract. It can contain variables, functions etc that perform different tasks. In a sense, a contract acts like a class. So, a basic contract to count and store the number of tasks in your to-do list would look like:

Here, "contract" is a keyword that signifies the contract. The name "TodoList" is the same as the name of

the file. "uint" is a type of state variable. "public" is the access modifier used to signify that this variable is available outside the function that it is declared in. Believe it or not, this is a simple

smart contract. This is it, you've written your first snippet of code in solidity. In your command line, type `truffle compile` to compile your solidity program. Of course, there are a few more steps and files required to deploy this to your local blockchain.

DEPLOYING A SMART CONTRACT

Your project will need some instructions in the file called `truffle-config.js`, as this is the "main" (i.e. the first) file that the server will detect on deployment. As of now, it is filled with some basic prewritten configurations. In order to deploy your smart contract to the right local network offered by Ganache, you have to replace that with this code: <https://pastebin.com/OevdcLZr>

In the `networks{...}` block of code, the host and port directly connect to Ganache, which helps us run the blockchain locally. Make sure your Ganache is running simultaneously with the project. Effectively, `truffle-config.js` will help you talk to your local blockchain. In order to get the smart contract onto the blockchain, you will have to create a migrations file.

Inside the "migrations" directory, you will notice that there is a file named `1_initial_migration.js`. You have to copy the contents of that file and create another one in the same directory called `2_deploy_contracts.js`. The naming configuration of these migration files is important. The number in front of the file name indicates the order in which truffle has to execute the migration files. Now, why do we need these migrations files?